Payment Provider Integration Family

Abstract Factory refactor of the checkout payment integration · Stripe, Adyen, Mercado Pago

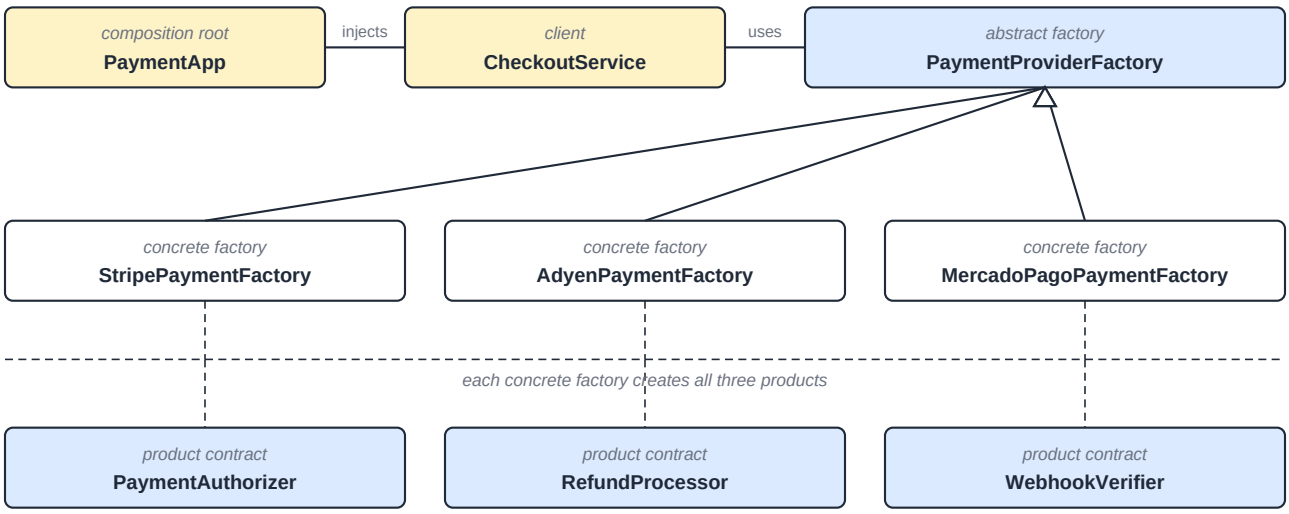
1. Roles checkout needs

Checkout only needs three capabilities from a payment provider. Each one is an abstract base class, and checkout talks to these contracts, never to a provider class.

Role	Contract	Method
Authorize a payment	`PaymentAuthorizer`	`authorize(amount) -> str`
Refund a payment	`RefundProcessor`	`refund(amount) -> str`
Verify a webhook	`WebhookVerifier`	`verify(signature) -> bool`

2. Pattern: Abstract Factory

`PaymentProviderFactory` is the single creation boundary for a complete provider family. `CheckoutService` receives a factory through its constructor (dependency injection) and builds its collaborators from it (composition). `PaymentApp` is the composition root: the only class that knows the concrete factories, using `PaymentProviderSelector` to turn a configured provider name into a factory.



Each concrete factory creates one implementation of every product contract,
e.g. MercadoPagoPaymentFactory -> MercadoPagoAuthorizer, MercadoPagoRefundProcessor, MercadoPagoWebhookVerifier.

```
PaymentProviderFactory (ABC)
  create_authorizer()        -> PaymentAuthorizer
  create_refund_processor()  -> RefundProcessor
  create_webhook_verifier()  -> WebhookVerifier

StripePaymentFactory       -> StripeAuthorizer, StripeRefundProcessor, StripeWebhookVerifier
AdyenPaymentFactory        -> AdyenAuthorizer, AdyenRefundProcessor, AdyenWebhookVerifier
MercadoPagoPaymentFactory  -> MercadoPagoAuthorizer, MercadoPagoRefundProcessor,
                              MercadoPagoWebhookVerifier
```

3. Design questions

What varies?

The provider: how each one authorizes, refunds and verifies webhooks. Concretely, the three product classes and the factory that builds them.

What stays stable?
The checkout business logic, the three role contracts and the factory contract. Checkout always authorizes, refunds and verifies the same way regardless of provider.

Why is this a family-level creation problem?
The three products are not independent. A Stripe authorizer must never be paired with an Adyen webhook verifier: payments, refunds and webhook signatures all belong to the same provider account. The original code already picked all three together inside an if/elif. The pressure is to create a consistent set of related objects, not a single object.

Which creational pattern did we choose, and why?
Abstract Factory. Factory Method fits when one product varies; here three related products vary together. One factory object per provider guarantees the family stays consistent, and swapping the factory swaps the whole family at once. Builder does not fit (there is no step-by-step construction of a complex object), and neither do Prototype or Singleton.

Who selects the concrete provider family?
The composition root (PaymentApp + PaymentProviderSelector), from configuration. Checkout never selects; it only receives a factory.

How was Mercado Pago added?
Three new product classes and one new factory, plus one entry in the composition root. `checkout_service.py` was not touched, and a test checks that it does not reference any provider name.

What complexity cost does the design introduce?
- More files and classes: 3 product contracts + 1 factory contract + 4 classes per provider. The starter had 7 classes; this version has 19.
- Indirection: following a call from checkout to the real provider code takes more hops.
- Adding a new role (for example, payouts) is expensive: the factory contract changes and every concrete factory has to implement it. Abstract Factory makes adding families easy and adding products hard.
- For two providers with trivial classes this is arguably more structure than needed; it pays off as providers and capabilities grow.

4. Tests

- Each family (Stripe, Adyen, Mercado Pago) works end to end through checkout.
- Every factory returns products that honor each contract and belong to the same provider.
- Checkout works with a fake factory, proving it depends only on the contract.
- checkout_service.py contains no provider names.
- Incomplete factories and unknown providers are rejected.

Run with `pytest -q` (18 tests).